



Training Guide — Part 3

Chapters 12 · 13 · 14

Object-Oriented Programming · Constructors & Destructors · Friend Functions

C++ OBJECT-ORIENTED PROGRAMMING – COMPLETE GUIDE

12.0 OOP Concepts: Classes, Objects, Principles

13.0 Constructors, Destructors, this Pointer, Static Members

14.0 Friend Functions and Friend Classes



Table of Contents

- [12.0 Object-Oriented Programming Concepts](#)
- [12.1 Classes and Objects](#)
 - [Classes vs Structs](#)
 - [Data Encapsulation](#)
 - [Abstraction](#)
 - [Polymorphism](#)
 - [Inheritance](#)
 - [Reusability](#)
 - [Composition vs Aggregation vs Inheritance](#)
 - [Static Members](#)
 - [Access Specifiers](#)
 - [Member Functions](#)
 - [Task 1Z](#)
- [13.0 Constructors, Destructors, this Pointer](#)
- [13.1 Constructors](#)
 - [Default Constructor](#)

- Parameterized Constructor
- Copy Constructor — Shallow & Deep Copy
- 13.2 Destructors
- 13.3 this Pointer
- 13.4 Static Variables in C and C++
- 13.5 Static Members in a Class
 - Task 2
- 14.0 Friend Functions and Friend Classes
- 14.1 Friend Functions
- 14.2 Friend Classes

12.0 Object-Oriented Programming Concepts

Learning Objectives

- Understand the paradigm shift from procedural to object-oriented thinking
- Define the four pillars of OOP: Encapsulation, Abstraction, Polymorphism, Inheritance
- Distinguish between a class (blueprint) and an object (instance)
- Know when to use composition, aggregation, or inheritance
- Apply access specifiers and member functions correctly

Introduction: From C to C++ — A New Way of Thinking

In procedural C programming, you write a sequence of instructions: do this, then do that. Data (variables) and behavior (functions) live separately. As programs grow large, this becomes hard to manage — you lose track of which functions are allowed to modify which data.

Object-Oriented Programming (OOP) solves this by bundling data and behavior together into self-contained units called **objects**. An object knows its own state (data) and knows how to behave (functions).

PROCEDURAL (C style):

```
Data:      float balance = 1000.0;
Functions: deposit(balance, 500);
           withdraw(balance, 200);
           printBalance(balance);
```

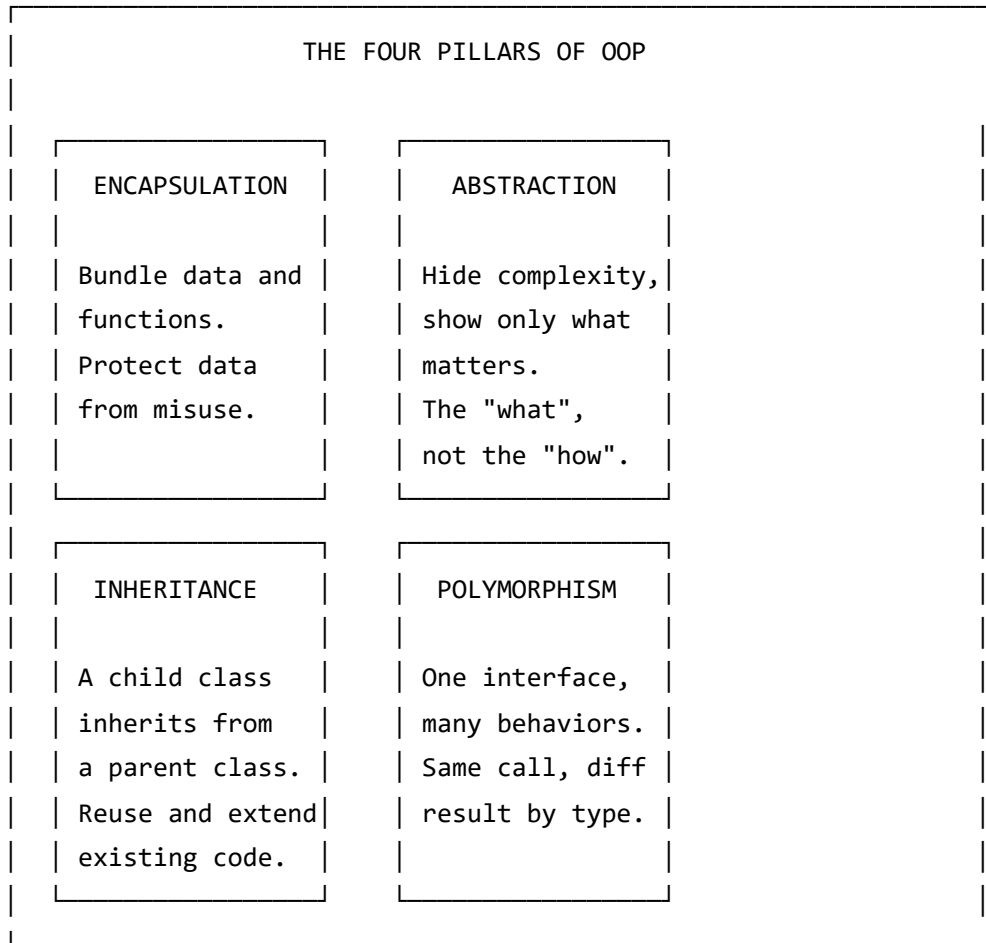
Problem: Any function in the program can touch 'balance' directly.
Nothing stops buggy code from setting balance = -99999.

OBJECT-ORIENTED (C++ style):

```
class BankAccount {
    float balance;      ← Data HIDDEN inside the object
public:
    void deposit(float); ← Only these approved functions
    void withdraw(float); can access balance
    void print();
};
```

```
BankAccount acc;
acc.deposit(500);  ← Clean, safe, clear
acc.withdraw(200);
```

The Four Pillars of OOP



12.1 Classes and Objects

Classes vs Structs

In C, a `struct` is a collection of related variables. In C++, both `struct` and `class` can hold data AND functions. The only mechanical difference is the **default access level**.

C STRUCT (data only, all public):

```
struct Point {  
    int x;  
    int y;  
};  
// Both x and y are directly accessible everywhere – no protection.
```

C++ STRUCT vs CLASS:

```
struct MyStruct {          class MyClass {  
    int data;    // PUBLIC    int data;    // PRIVATE by default  
    void show(); // PUBLIC    void show(); // PRIVATE by default  
};                  };
```

STRUCT	→ default access is PUBLIC
CLASS	→ default access is PRIVATE

In practice: use struct for plain data (no logic), use class for objects with behavior.
--

Side-by-Side Comparison

Feature	C Struct	C++ Struct	C++ Class
Default access	Public	Public	Private
Member functions	No	Yes	Yes
Constructors	No	Yes	Yes
Inheritance	No	Yes	Yes
Access control	No	Yes	Yes
Convention	Plain data	Simple data+func	Full OOP

```

#include <stdio.h>

// — C-STYLE STRUCT (no protection) —————
struct PointC {
    int x;
    int y;
};

// — C++ CLASS (with protection and behavior) —————
class PointCPP {
private:                // Data hidden from outside
    int x;
    int y;

public:                // Interface available to the world
    void set(int px, int py) { x = px; y = py; }
    void print() { printf("Point(%d, %d)\n", x, y); }
    float distanceFromOrigin() {
        return (float)((x * x) + (y * y)); // simplified
    }
};

int main() {
    // C-style: direct access (risky)
    PointC p1;
    p1.x = 3; p1.y = 4;                // Anyone can set x,y
    printf("C struct: x=%d, y=%d\n", p1.x, p1.y);

    // C++ class: controlled access (safe)
    PointCPP p2;
    p2.set(3, 4);                      // Must go through set()
    p2.print();
    // p2.x = 99; ← compile error! x is private.

    return 0;
}

```

Access Specifiers

Access specifiers control **who can see what**. They are the gatekeepers of encapsulation.

ACCESS SPECIFIERS	
PRIVATE:	Only this class's own member functions. • Most restrictive. • Default in class. • Use for: internal data, helper functions.
PROTECTED:	This class AND its derived (child) classes. • Middle ground. • Use for: data child classes need to inherit.
PUBLIC:	Everyone – any code anywhere. • Most permissive. • Use for: the class's official interface.

ACCESS DIAGRAM:

	Own Class	Derived Class	Outside Code
private:	✓	X	X
protected:	✓	✓	X
public:	✓	✓	✓

```

class Employee {
private:
    float salary;          // Only Employee methods touch this
    int    ssn;            // Sensitive: completely hidden

protected:
    char    department[30]; // Child classes (Manager) can access this

public:
    char    name[50];       // Public: anyone can read the name
    void    setSalary(float s); // Public setter – controlled access
    float    getSalary();    // Public getter – read-only access
};

class Manager : public Employee {
    void promote() {
        // department is accessible here (protected inheritance)
        // salary is NOT accessible here (private to Employee)
    }
};

```

Data Encapsulation

Encapsulation means **bundling data and the functions that operate on it** into a single unit (the class), and **restricting direct access** to the data from outside.

Think of a capsule (pill) — the medicine is enclosed and protected. You can't directly touch the chemicals inside; you interact through the outer shell.

WITHOUT ENCAPSULATION:

```
float balance = 1000;  
balance = -99999; ← any code!
```

WITH ENCAPSULATION:

```
class Account {  
private:  
    float balance;  
public:  
    void deposit(float amt) {  
        if (amt > 0)  
            balance += amt;  
    }  
    void withdraw(float amt) {  
        if (amt > 0 && amt <= balance)  
            balance -= amt;  
    }  
};  
// balance = -99999 is IMPOSSIBLE
```

Getters and Setters

The standard pattern for encapsulation is **private data + public getters/setters**:

```

#include <stdio.h>
#include <string.h>

class Student {
private:
    // — PRIVATE DATA: hidden, protected —————
    char name[50];
    int age;
    float gpa;

public:
    // — SETTER: validates before assigning —————
    void setName(const char* n) {
        if (n != NULL && strlen(n) > 0) {
            strncpy(name, n, 49);
            name[49] = '\0';
        } else {
            printf("Error: invalid name.\n");
        }
    }

    void setAge(int a) {
        if (a >= 5 && a <= 100)
            age = a;
        else
            printf("Error: age must be 5-100.\n");
    }

    void setGPA(float g) {
        if (g >= 0.0f && g <= 4.0f)
            gpa = g;
        else
            printf("Error: GPA must be 0.0-4.0.\n");
    }

    // — GETTERS: read-only access —————
    const char* getName() const { return name; }
    int getAge() const { return age; }
    float getGPA() const { return gpa; }

    // — BEHAVIOR: member function —————
    void print() const {
        printf("Student: %-20s | Age: %2d | GPA: %.2f\n",

```

```

        name, age, gpa);
    }

    char* getHonors() const {
        if (gpa >= 3.9f)      return (char*)"Summa Cum Laude";
        else if (gpa >= 3.7f) return (char*)"Magna Cum Laude";
        else if (gpa >= 3.5f) return (char*)"Cum Laude";
        else                  return (char*)"";
    }
};

int main() {
    Student s;
    s.setName("Alice Johnson");
    s.setAge(20);
    s.setGPA(3.92f);
    s.print();
    printf("Honors: %s\n\n", s.getHonors());

    // Validation in action:
    s.setAge(500);      // Error: age must be 5-100.
    s.setGPA(5.0f);     // Error: GPA must be 0.0-4.0.

    // s.age = 500;    ← Compile error! private member.

    return 0;
}

```

Output:

```

Student: Alice Johnson      | Age: 20 | GPA: 3.92
Honors: Summa Cum Laude

```

```

Error: age must be 5-100.
Error: GPA must be 0.0-4.0.

```

Key Principles

Abstraction

Abstraction means **hiding complexity and showing only what is essential**. The user of a class sees a clean interface (what it does) without needing to understand the internal implementation (how it does it).

ABSTRACTION ANALOGY: A Car

What the driver sees (interface):

- Steering wheel
- Accelerator pedal
- Brake pedal
- Gear shift

What the driver does NOT need to know:

- How the combustion engine works
- How fuel injection is timed
- How the braking hydraulics operate

The car ABSTRACTS the complexity.

The driver uses a simple interface to control a complex machine.

```
// ABSTRACTION: the user calls sort(), not knowing the algorithm
class DataProcessor {
private:
    int data[1000];
    int size;

    // PRIVATE: implementation detail hidden from user
    void bubbleSortInternal() {
        for (int i = 0; i < size - 1; i++)
            for (int j = 0; j < size - 1 - i; j++)
                if (data[j] > data[j+1]) {
                    int t = data[j]; data[j] = data[j+1]; data[j+1] = t;
                }
    }

    void mergeSortInternal(int left, int right) {
        // ... complex merge sort implementation ...
    }

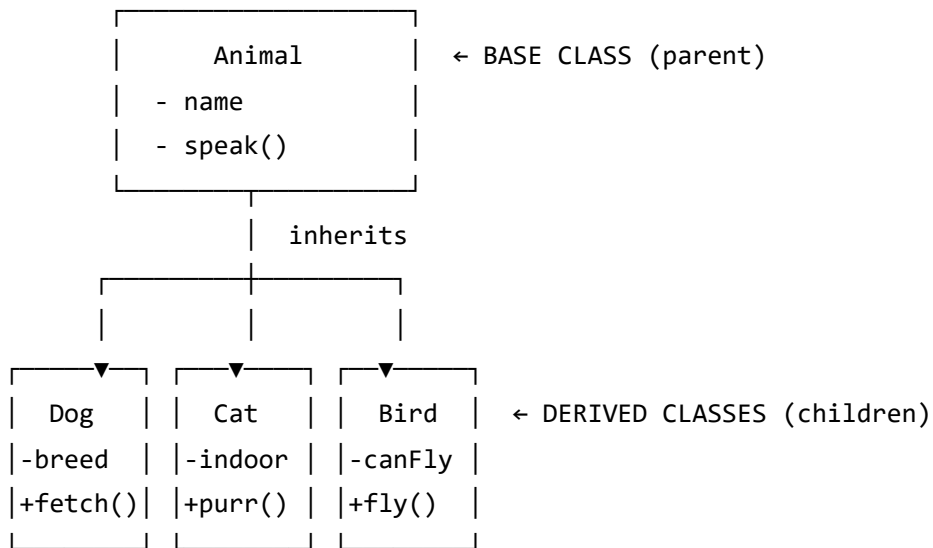
public:
    // PUBLIC: clean, simple interface – user doesn't care HOW it sorts
    void addData(int val) { if (size < 1000) data[size++] = val; }
    void sort()           { bubbleSortInternal(); } // could swap algorithm
    int  getMin()          { return data[0]; }       // assumes sorted
    int  getMax()          { return data[size - 1]; }
    void printAll();
};

// The user:
DataProcessor dp;
dp.addData(50); dp.addData(10); dp.addData(30);
dp.sort();      // User doesn't need to know IT'S bubble sort
int min = dp.getMin(); // Clean. Simple. Abstract.
```

Inheritance

Inheritance lets a **child class** (derived class) acquire all the properties and behaviors of a **parent class** (base class), then **extend or override** them.

INHERITANCE HIERARCHY:



Dog IS-A Animal. Cat IS-A Animal.

Each inherits name + speak() from Animal.

Each adds its own unique properties and behaviors.

```

#include <stdio.h>
#include <string.h>

// — BASE CLASS —————
class Animal {
protected:
    char name[30];
    int age;

public:
    void setName(const char* n) { strncpy(name, n, 29); }
    void setAge(int a)          { age = a; }
    const char* getName()       { return name; }

    // Virtual: can be OVERRIDDEN by child classes
    virtual void speak() {
        printf("%s makes a sound.\n", name);
    }

    void breathe() {
        printf("%s breathes oxygen.\n", name);
    }
};

// — DERIVED CLASS: Dog inherits from Animal —————
class Dog : public Animal {
private:
    char breed[30];

public:
    void setBreed(const char* b) { strncpy(breed, b, 29); }

    // OVERRIDE parent's speak()
    void speak() override {
        printf("%s says: Woof! Woof!\n", name);
    }

    // NEW behavior specific to Dog
    void fetch() {
        printf("%s fetches the ball!\n", name);
    }

    void print() {

```

```

        printf("Dog: %s | Breed: %s | Age: %d\n", name, breed, age);
    }
};

// ——— DERIVED CLASS: Cat inherits from Animal —————
class Cat : public Animal {
private:
    int indoor;    // 1=indoor, 0=outdoor

public:
    void setIndoor(int i) { indoor = i; }

    void speak() override {
        printf("%s says: Meow!\n", name);
    }

    void purr() {
        printf("%s purrs contentedly...\n", name);
    }
};

int main() {
    Dog d;
    d.setName("Rex");
    d.setAge(3);
    d.setBreed("German Shepherd");
    d.print();
    d.speak();    // Dog's speak (overridden)
    d.breathe();  // Inherited from Animal
    d.fetch();    // Dog-specific

    printf("\n");

    Cat c;
    c.setName("Whiskers");
    c.setAge(5);
    c.setIndoor(1);
    c.speak();    // Cat's speak (overridden)
    c.breathe();  // Inherited from Animal
    c.purr();     // Cat-specific

    return 0;
}

```

Output:

```
Dog: Rex | Breed: German Shepherd | Age: 3
Rex says: Woof! Woof!
Rex breathes oxygen.
Rex fetches the ball!

Whiskers says: Meow!
Whiskers breathes oxygen.
Whiskers purrs contentedly...
```

Types of Inheritance

	INHERITANCE TYPES IN C++
SINGLE:	One parent → One child class Dog : public Animal
MULTIPLE:	Two+ parents → One child class Amphibian : public Land, public Water
MULTILEVEL:	Grandparent → Parent → Child class GoldenRetriever : public Dog (Dog already inherits from Animal)
HIERARCHICAL:	One parent → Many children class Dog : Animal, Cat : Animal, Bird : Animal
HYBRID:	Combination of above

Inheritance Access Levels

Base member	public inherit	protected inherit	private inherit
public:	public	protected	private
protected:	protected	protected	private
private:	inaccessible	inaccessible	inaccessible

Polymorphism

Polymorphism means **"many forms"** — the same function call behaves differently depending on the actual object type at runtime.

POLYMORPHISM IN ACTION:

```
Animal* animals[3];
animals[0] = new Dog();    // Dog stored as Animal pointer
animals[1] = new Cat();
animals[2] = new Bird();

for (int i = 0; i < 3; i++) {
    animals[i]-> speak();  // SAME CALL → different behavior!
}
```

Output:

```
Rex says: Woof! Woof!    ← Dog's speak()
Whiskers says: Meow!     ← Cat's speak()
Tweety says: Tweet!      ← Bird's speak()
```

ONE interface (speak), MANY behaviors.

This works through VIRTUAL FUNCTIONS.

```

#include <stdio.h>

class Shape {
public:
    // virtual: enables polymorphism (runtime dispatch)
    // = 0: makes this a PURE VIRTUAL function (abstract)
    //      Shape cannot be instantiated directly
    virtual float area() = 0;
    virtual void print() = 0;

    // Non-virtual: shared behavior (not overridable)
    void describe() {
        printf("I am a shape. My area is %.2f\n", area());
    }
};

class Circle : public Shape {
    float radius;
public:
    Circle(float r) : radius(r) {}

    float area() override {
        return 3.14159f * radius * radius;
    }

    void print() override {
        printf("Circle (radius=%.1f) area=%.2f\n", radius, area());
    }
};

class Rectangle : public Shape {
    float width, height;
public:
    Rectangle(float w, float h) : width(w), height(h) {}

    float area() override {
        return width * height;
    }

    void print() override {
        printf("Rectangle (%.1f x %.1f) area=%.2f\n", width, height, area());
    }
};

```

```

class Triangle : public Shape {
    float base, height;
public:
    Triangle(float b, float h) : base(b), height(h) {}

    float area() override { return 0.5f * base * height; }

    void print() override {
        printf("Triangle (base=%.1f, h=%.1f) area=%.2f\n",
            base, height, area());
    }
};

int main() {
    // Polymorphism: array of Shape pointers, each holding a different type
    Shape* shapes[3];
    shapes[0] = new Circle(5.0f);
    shapes[1] = new Rectangle(4.0f, 6.0f);
    shapes[2] = new Triangle(3.0f, 8.0f);

    printf("=== Polymorphism Demo ===\n\n");

    for (int i = 0; i < 3; i++) {
        shapes[i]->print();    // Different behavior, same call
        shapes[i]->describe(); // Calls area() polymorphically too
        printf("\n");
    }

    // Find total area – works for ANY shape!
    float total = 0;
    for (int i = 0; i < 3; i++)
        total += shapes[i]->area();
    printf("Total area: %.2f\n", total);

    // Cleanup
    for (int i = 0; i < 3; i++) delete shapes[i];

    return 0;
}

```

Output:

=== Polymorphism Demo ===

Circle (radius=5.0) area=78.54

I am a shape. My area is 78.54

Rectangle (4.0 x 6.0) area=24.00

I am a shape. My area is 24.00

Triangle (base=3.0, h=8.0) area=12.00

I am a shape. My area is 12.00

Total area: 114.54

 **Virtual Function Table (vtable):** When a class has virtual functions, the compiler creates a hidden table of function pointers (vtable). Each object has a pointer to its class's vtable. When you call `shape->area()`, the vtable routes the call to the correct override at runtime. This is called **dynamic dispatch**.

Reusability

OOP promotes code reuse through inheritance, composition, and templates. Once you write and test a class, you never need to rewrite it:

WITHOUT OOP (duplicate code everywhere):

```
void printDogInfo(char* name, int age, char* breed) { ... }
void printCatInfo(char* name, int age, int indoor) { ... }
void printBirdInfo(char* name, int age, float wingspan) { ... }
// Every new animal type = rewrite the common parts!
```

WITH OOP (write once, reuse everywhere):

```
class Animal {
    // name, age, breathe(), eat() written ONCE
};
class Dog : public Animal { /* add breed, fetch() */ };
class Cat : public Animal { /* add indoor, purr() */ };
class Bird : public Animal { /* add wingspan, fly() */ };
// New animal? Just inherit. Common logic = zero duplication.
```

Composition vs Aggregation vs Inheritance

These are three different ways classes can **relate to each other**. Choosing the right one is one of the most important OOP design decisions.

COMPOSITION vs AGGREGATION vs INHERITANCE		
COMPOSITION	AGGREGATION	INHERITANCE
"HAS-A (owns)"	"HAS-A (uses)"	"IS-A"
Part cannot exist without the whole.	Part can exist independently.	Child IS a type of the parent.
Heart HAS-A Human body. Heart dies when body dies.	Library HAS Books. Books exist without the Library.	Dog IS AN Animal. Manager IS AN Employee.
Strong "part-of" ownership.	Weak ownership.	Use when child is a specialization.

```

#include <stdio.h>
#include <string.h>

// — COMPOSITION: Engine "part of" Car (Engine dies with Car) —
class Engine {
    int horsepower;
public:
    Engine(int hp) : horsepower(hp) {}
    void start() { printf("Engine (%d HP) started.\n", horsepower); }
};

class Car {
    Engine engine;    // COMPOSITION: Engine is part OF Car
    char model[30];
public:
    Car(const char* m, int hp) : engine(hp) { // Engine created WITH Car
        strncpy(model, m, 29);
    }
    void drive() {
        engine.start();
        printf("%s is driving.\n", model);
    }
};

// — AGGREGATION: Library "uses" Books (Books exist independently) —
class Book {
    char title[50];
public:
    Book(const char* t) { strncpy(title, t, 49); }
    const char* getTitle() { return title; }
};

class Library {
    Book* books[100];    // AGGREGATION: pointers to existing Books
    int count;
public:
    Library() : count(0) {}
    void addBook(Book* b) { if (count < 100) books[count++] = b; }
    void listBooks() {
        printf("Library contains %d books:\n", count);
        for (int i = 0; i < count; i++)
            printf("  - %s\n", books[i]->getTitle());
    }
}

```

```

};

// — INHERITANCE: Manager IS-AN Employee —————
class Employee {
protected:
    char name[30];
    float salary;
public:
    Employee(const char* n, float s) : salary(s) { strncpy(name, n, 29); }
    virtual void work() { printf("%s works.\n", name); }
};

class Manager : public Employee {           // Manager IS AN Employee
    int teamSize;
public:
    Manager(const char* n, float s, int ts)
        : Employee(n, s), teamSize(ts) {}
    void work() override {
        printf("%s manages a team of %d.\n", name, teamSize);
    }
};

int main() {
    // COMPOSITION
    printf("=== Composition ===\n");
    Car myCar("Toyota Camry", 203);
    myCar.drive();

    // AGGREGATION
    printf("\n=== Aggregation ===\n");
    Book b1("Clean Code"), b2("CLRS Algorithms"), b3("The C Language");
    Library lib;
    lib.addBook(&b1); lib.addBook(&b2); lib.addBook(&b3);
    lib.listBooks();
    // Books b1, b2, b3 still exist even after lib goes out of scope

    // INHERITANCE
    printf("\n=== Inheritance ===\n");
    Employee* staff[2];
    staff[0] = new Employee("Alice", 60000);
    staff[1] = new Manager("Bob", 90000, 8);
    for (int i = 0; i < 2; i++) staff[i]->work();
    delete staff[0]; delete staff[1];
}

```

```
    return 0;
}
```

Output:

```
=== Composition ===
Engine (203 HP) started.
Toyota Camry is driving.
```

```
=== Aggregation ===
Library contains 3 books:
- Clean Code
- CLRS Algorithms
- The C Language
```

```
=== Inheritance ===
Alice works.
Bob manages a team of 8.
```

Static Members

Static members belong to the **class itself**, not to any individual object. All objects of the class share the same static member.

NON-STATIC (instance variable):	STATIC (class variable):
Object 1: name = "Alice"	All objects share:
Object 2: name = "Bob"	count = 3 (total objects)
Object 3: name = "Carol"	
Each has its OWN name.	There is only ONE count.

```

class Counter {
private:
    int id;
    static int objectCount;    // Shared across ALL Counter objects

public:
    Counter() {
        objectCount++;
        id = objectCount;
        printf("Counter #%d created. Total: %d\n", id, objectCount);
    }

    static int getCount() {    // Static function: no 'this' pointer
        return objectCount;
    }
};

// MUST define static member outside the class
int Counter::objectCount = 0;

int main() {
    Counter c1; // Counter #1 created. Total: 1
    Counter c2; // Counter #2 created. Total: 2
    Counter c3; // Counter #3 created. Total: 3

    // Can call static function on the CLASS (no object needed)
    printf("Total counters: %d\n", Counter::getCount());
}

```

Member Functions

Member functions (methods) define the **behavior** of a class. They have full access to all class members (private, protected, public).

TYPES OF MEMBER FUNCTIONS:

Regular member function	→ Has 'this' pointer, accesses instance data
const member function	→ Promises not to modify the object
static member function	→ No 'this' pointer, works with static data
virtual member function	→ Can be overridden by derived classes
pure virtual function	→ MUST be overridden, makes class abstract
inline member function	→ Defined inside class body, hint to compiler

```

class Rectangle {
private:
    float width;
    float height;

public:
    // — REGULAR: modifies object —————
    void setDimensions(float w, float h) {
        width = w; height = h;
    }

    // — CONST: read-only, cannot modify the object —————
    float area() const { return width * height; }
    float perimeter() const { return 2 * (width + height); }

    // — INLINE: defined in class body (compiler may inline it) —
    bool isSquare() const { return width == height; }

    // — DEFINED OUTSIDE THE CLASS —————
    void print() const; // Declaration only

    // — STATIC: no 'this', no access to width/height —————
    static Rectangle createSquare(float side) {
        Rectangle r;
        r.width = r.height = side;
        return r;
    }
};

// Definition outside the class using the :: scope resolution operator
void Rectangle::print() const {
    printf("Rectangle(%.1f x %.1f) - area=%.2f, perimeter=%.2f, %s\n",
        width, height, area(), perimeter(),
        isSquare() ? "SQUARE" : "not square");
}

int main() {
    Rectangle r;
    r.setDimensions(4.0f, 6.0f);
    r.print();

    Rectangle sq = Rectangle::createSquare(5.0f);

```

```
sq.print();  
}
```

Output:

```
Rectangle(4.0 x 6.0) - area=24.00, perimeter=20.00, not square  
Rectangle(5.0 x 5.0) - area=25.00, perimeter=20.00, SQUARE
```

Complete Class Example: BankAccount

Putting it all together — a production-quality class demonstrating all principles:

```

#include <stdio.h>
#include <string.h>

class BankAccount {
private:
    // — PRIVATE DATA —————
    char    owner[50];
    double  balance;
    int     accountNumber;

    static int nextAccountNumber; // Shared counter for unique IDs
    static int totalAccounts;     // Track all accounts

    // — PRIVATE HELPER —————
    bool isValidAmount(double amount) const {
        return amount > 0.0;
    }

public:
    // — CONSTRUCTOR —————
    BankAccount(const char* ownerName, double initialBalance) {
        strncpy(owner, ownerName, 49);
        balance = (initialBalance >= 0) ? initialBalance : 0.0;
        accountNumber = ++nextAccountNumber;
        totalAccounts++;
        printf("Account #%04d opened for %s (balance: $%.2f)\n",
            accountNumber, owner, balance);
    }

    // — PUBLIC OPERATIONS —————
    bool deposit(double amount) {
        if (!isValidAmount(amount)) {
            printf("Error: deposit amount must be positive.\n");
            return false;
        }
        balance += amount;
        printf("[#%04d] Deposited $%.2f. New balance: $%.2f\n",
            accountNumber, amount, balance);
        return true;
    }

    bool withdraw(double amount) {
        if (!isValidAmount(amount)) {

```

```

        printf("Error: withdraw amount must be positive.\n");
        return false;
    }
    if (amount > balance) {
        printf("Error: insufficient funds. Balance: $%.2f\n", balance);
        return false;
    }
    balance -= amount;
    printf("[#%04d] Withdrew $%.2f. New balance: $%.2f\n",
        accountNumber, amount, balance);
    return true;
}

// — CONST GETTERS —————
double    getBalance() const { return balance; }
int        getAccountNumber() const { return accountNumber; }
const char* getOwner() const { return owner; }

// — STATIC METHOD —————
static int getTotalAccounts() { return totalAccounts; }

void printStatement() const {
    printf("┌───────────────────────────────────┐\n");
    printf("│ Account #04d │\n", accountNumber);
    printf("│ Owner:   %-26s │\n", owner);
    printf("│ Balance: $%-25.2f │\n", balance);
    printf("└───────────────────────────────────┘\n");
}

};

// Static member definitions
int BankAccount::nextAccountNumber = 1000;
int BankAccount::totalAccounts     = 0;

int main() {
    printf("=== Bank Account OOP Demo ===\n\n");

    BankAccount alice("Alice Smith", 500.00);
    BankAccount bob("Bob Johnson", 1200.00);

    printf("\n");
    alice.deposit(250.00);
    alice.withdraw(100.00);

```

```

    alice.withdraw(700.00);    // Should fail

    printf("\n");
    bob.deposit(300.00);
    bob.withdraw(500.00);

    printf("\n");
    alice.printStatement();
    bob.printStatement();

    printf("\nTotal bank accounts: %d\n", BankAccount::getTotalAccounts());

    return 0;
}

```

Output:

=== Bank Account OOP Demo ===

Account #1001 opened for Alice Smith (balance: \$500.00)

Account #1002 opened for Bob Johnson (balance: \$1200.00)

[#1001] Deposited \$250.00. New balance: \$750.00

[#1001] Withdrew \$100.00. New balance: \$650.00

Error: insufficient funds. Balance: \$650.00

[#1002] Deposited \$300.00. New balance: \$1500.00

[#1002] Withdrew \$500.00. New balance: \$1000.00

Account #1001
Owner: Alice Smith
Balance: \$650.00

Account #1002
Owner: Bob Johnson
Balance: \$1000.00

Total bank accounts: 2

Task 1Z

Design and implement a Library management system using C++ OOP.

Requirements:

1. Create a `Book` class with: title, author, ISBN, available copies. Include full getters/setters with validation.
2. Create a `Member` class with: name, member ID, list of borrowed books (max 3).
3. Create a `Library` class (composition of Books and Members) with methods: `addBook()` , `addMember()` , `borrowBook()` , `returnBook()` , `displayAll()` .
4. Apply: Encapsulation, Abstraction, Static member for unique IDs.
5. Demonstrate with: 3 books, 2 members, borrow and return operations.

Bonus: Add a `PremiumMember` class (inherits `Member`) that can borrow up to 5 books. Demonstrate polymorphism.

13.0 Constructors, Destructors, this Pointer

Learning Objectives

- Understand when and why constructors and destructors are automatically called
- Implement default, parameterized, and copy constructors
- Distinguish between shallow copy and deep copy — and know when each is dangerous
- Use the `this` pointer to resolve ambiguity and enable method chaining
- Understand static variables in both C and C++ class contexts

Introduction: Automatic Object Lifecycle Management

One of C++'s most powerful features is **automatic resource management**. When an object is created, its constructor runs automatically. When it goes out of scope or is deleted, its destructor runs

automatically.

OBJECT LIFECYCLE:

```
{                                ← scope begins
    BankAccount acc("Alice");  ← CONSTRUCTOR called automatically
    acc.deposit(500);
    acc.withdraw(100);
}
```

No manual "initialize" or "cleanup" calls needed!

The constructor guarantees a valid initial state.

The destructor guarantees proper cleanup.

13.1 Constructors

A **constructor** is a special member function that:

- Has the **same name as the class**
- Has **no return type** (not even void)
- Is called **automatically** when an object is created
- Can be **overloaded** (multiple constructors with different parameters)

TYPES OF CONSTRUCTORS	
Default Constructor	→ no parameters
Parameterized Constructor	→ takes arguments
Copy Constructor	→ creates a copy of another object

Default Constructor

A **default constructor** takes no parameters. It gives the object a known, safe initial state.

```

#include <stdio.h>
#include <string.h>

class Temperature {
private:
    float celsius;
    char unit;          // 'C', 'F', or 'K'

public:
    // ——— DEFAULT CONSTRUCTOR —————
    // Called when you write: Temperature t;
    Temperature() {
        celsius = 0.0f;    // Initialize to known values
        unit = 'C';
        printf("Default constructor: Temperature set to %.1f°C\n", celsius);
    }

    // Getters
    float getCelsius() const { return celsius; }
    float getFahrenheit() const { return celsius * 9.0f / 5.0f + 32.0f; }
    float getKelvin() const { return celsius + 273.15f; }
    char getUnit() const { return unit; }

    void print() const {
        printf("Temperature: %.2f°C / %.2f°F / %.2fK\n",
            getCelsius(), getFahrenheit(), getKelvin());
    }
};

int main() {
    Temperature t1;          // Default constructor called
    t1.print();

    Temperature t2;          // Another default constructor call
    t2.print();

    // Without default constructor, t1.celsius would be GARBAGE
    // (uninitialized memory – a very common bug in C)

    return 0;
}

```

Output:

Default constructor: Temperature set to 0.0°C

Temperature: 0.00°C / 32.00°F / 273.15K

Default constructor: Temperature set to 0.0°C

Temperature: 0.00°C / 32.00°F / 273.15K

 **Important:** If you define ANY constructor (e.g., a parameterized one), the compiler stops providing a free default constructor. You must explicitly define it if you still want `MyClass obj;` to work.

Parameterized Constructor

A **parameterized constructor** accepts arguments to initialize the object with specific values at creation time.

```

#include <stdio.h>
#include <string.h>

class Vector3D {
private:
    float x, y, z;

public:
    // ——— DEFAULT CONSTRUCTOR ———
    Vector3D() : x(0.0f), y(0.0f), z(0.0f) {
        printf("Vector3D default: (0,0,0)\n");
    }

    // ——— PARAMETERIZED CONSTRUCTOR ———
    // Called when you write: Vector3D v(1.0, 2.0, 3.0);
    // Uses MEMBER INITIALIZER LIST (: x(px), y(py), z(pz))
    // This is more efficient than assigning in the body
    Vector3D(float px, float py, float pz) : x(px), y(py), z(pz) {
        printf("Vector3D parameterized: (%.1f, %.1f, %.1f)\n", x, y, z);
    }

    // ——— SINGLE-VALUE CONSTRUCTOR ———
    // Creates a vector where all components are equal
    explicit Vector3D(float all) : x(all), y(all), z(all) {
        // 'explicit' prevents accidental implicit conversion
        // Without it: Vector3D v = 5.0f; would work (bad!)
        // With it:    Vector3D v = 5.0f; → compile error (good!)
        printf("Vector3D uniform: (%.1f, %.1f, %.1f)\n", all, all, all);
    }

    float magnitude() const {
        return (float)((x*x + y*y + z*z)); // simplified (no sqrt)
    }

    Vector3D add(const Vector3D& other) const {
        return Vector3D(x + other.x, y + other.y, z + other.z);
    }

    void print() const {
        printf("Vector(%.2f, %.2f, %.2f)\n", x, y, z);
    }
};

```

```

int main() {
    printf("=== Parameterized Constructor Demo ===\n\n");

    Vector3D v1;                // Default:      (0, 0, 0)
    Vector3D v2(3.0f, 4.0f, 0.0f); // Parameterized: (3,4,0)
    Vector3D v3(5.0f);           // Uniform:      (5,5,5)

    printf("\n");
    v1.print();
    v2.print();
    v3.print();

    printf("\nv2 + v3 = ");
    v2.add(v3).print();

    return 0;
}

```

Member Initializer List

— BODY ASSIGNMENT (less efficient) —

```

Vector3D(float px, float py, float pz) {
    x = px;    // These are ASSIGNMENTS, not initializations.
    y = py;    // For built-in types: no difference.
    z = pz;    // For class-type members: member is default-constructed
}              // FIRST, then assigned – wasteful!

```

— INITIALIZER LIST (preferred, more efficient) —

```

Vector3D(float px, float py, float pz) : x(px), y(py), z(pz) {
    // x, y, z are DIRECTLY INITIALIZED.
    // For class-type members: NO default constructor + assignment,
    // just ONE direct initialization call.
}

```

REQUIRED for:

- ✓ const member variables (must be initialized, not assigned)
- ✓ reference member variables
- ✓ base class initialization
- ✓ Member objects without a default constructor

Copy Constructor — Shallow and Deep Copy

The **copy constructor** is called when an object is initialized from another object of the same class.

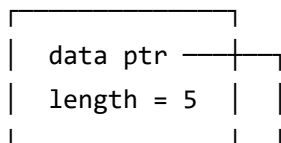
```
ClassName obj2 = obj1;           // Copy constructor
ClassName obj2(obj1);           // Equivalent
void func(ClassName obj);       // Copy when passed by value
ClassName func();               // Copy when returned by value
```

Shallow Copy — The Danger

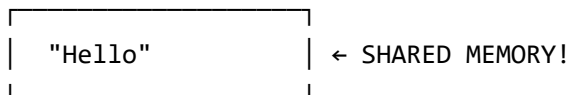
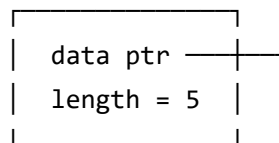
When your class contains **pointer members**, the default (compiler-generated) copy constructor does a **shallow copy** — it copies the pointer value (address), NOT the data it points to. Both objects then point to the **same memory** — a serious bug.

SHALLOW COPY (DANGEROUS with pointers):

Original:



Copy:



Problem 1: Copy modifies "Hello" → original is also changed!

Problem 2: Original's destructor frees this memory.

Copy's destructor tries to free it AGAIN → CRASH (double free)!

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

// — CLASS WITH SHALLOW COPY PROBLEM —————
class ShallowString {
public:
    char* data;

    ShallowString(const char* str) {
        data = (char*)malloc(strlen(str) + 1);
        strcpy(data, str);
        printf("ShallowString constructed: \"%s\"\n", data);
    }

    // NO copy constructor defined → compiler generates shallow copy
    // Both objects share the same data pointer!

    ~ShallowString() {
        printf("ShallowString destroyed: \"%s\"\n", data);
        free(data);    // DOUBLE FREE if there's a shallow copy!
    }
};

// — SHALLOW COPY DEMO —————
void shallowCopyDemo() {
    ShallowString s1("Hello");
    ShallowString s2 = s1;        // Shallow copy: s2.data = s1.data

    s2.data[0] = 'J';            // MODIFIES s1.data too!

    printf("s1: %s\n", s1.data);  // Prints "Jello"! Bug!
    printf("s2: %s\n", s2.data);  // Prints "Jello"

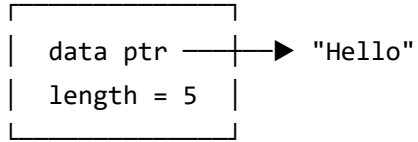
    // Both destructors free the SAME pointer → undefined behavior (crash)!
}

```

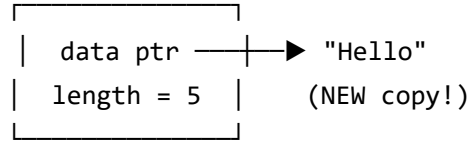
Deep Copy — The Solution

DEEP COPY (SAFE):

Original:



Copy:



Each object has its OWN memory.

Changes to one don't affect the other.

Each destructor frees its own memory safely.

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

// — CLASS WITH DEEP COPY —————
class DeepString {
private:
    char* data;
    int length;

public:
    // — REGULAR CONSTRUCTOR —————
    DeepString(const char* str) {
        length = strlen(str);
        data = (char*)malloc(length + 1);
        strcpy(data, str);
        printf("Constructed: \"%s\" (at address %p)\n", data, (void*)data);
    }

    // — DEEP COPY CONSTRUCTOR —————
    // MUST allocate NEW memory and copy the CONTENTS, not the pointer!
    DeepString(const DeepString& other) {
        length = other.length;
        data = (char*)malloc(length + 1); // ← NEW allocation!
        strcpy(data, other.data);         // ← Copy CONTENTS
        printf("Deep copied: \"%s\" (at NEW address %p)\n",
            data, (void*)data);
    }

    // — COPY ASSIGNMENT OPERATOR —————
    // Called for: existing_obj = other_obj;
    DeepString& operator=(const DeepString& other) {
        if (this == &other) return *this; // Self-assignment check!

        free(data); // Free old memory
        length = other.length;
        data = (char*)malloc(length + 1); // New memory
        strcpy(data, other.data); // Copy contents

        printf("Assigned: \"%s\"\n", data);
        return *this;
    }
}

```

```

// — DESTRUCTOR —————
~DeepString() {
    printf("Destroyed: \"%s\" (freed %p)\n", data, (void*)data);
    free(data);    // Safe: each object owns its own memory
}

void append(char ch) {
    char* newData = (char*)malloc(length + 2);
    strcpy(newData, data);
    newData[length] = ch;
    newData[length+1] = '\0';
    free(data);
    data = newData;
    length++;
}

void print() const {
    printf("DeepString: \"%s\" (length=%d)\n", data, length);
}

};

int main() {
    printf("=== Deep Copy Constructor Demo ===\n\n");

    DeepString s1("Hello");
    DeepString s2 = s1;    // Deep copy constructor called

    printf("\n--- Modifying s2 (should NOT affect s1) ---\n");
    s2.append('!');

    printf("\n");
    s1.print();    // Still "Hello"
    s2.print();    // Now "Hello!"

    printf("\n--- Copy assignment ---\n");
    DeepString s3("World");
    s3 = s1;    // Copy assignment operator called

    s3.print();

    printf("\n--- Destructors ---\n");
    // s1, s2, s3 go out of scope – each frees its own memory safely

```

```
    return 0;  
}
```

Output:

=== Deep Copy Constructor Demo ===

Constructed: "Hello" (at address 0x55a1c1234)

Deep copied: "Hello" (at NEW address 0x55a1c1258)

--- Modifying s2 (should NOT affect s1) ---

DeepString: "Hello" (length=5)

DeepString: "Hello!" (length=6)

--- Copy assignment ---

Constructed: "World" (at address 0x55a1c1290)

Assigned: "Hello"

DeepString: "Hello" (length=5)

--- Destructors ---

Destroyed: "Hello" (freed 0x55a1c1290)

Destroyed: "Hello!" (freed 0x55a1c1258)

Destroyed: "Hello" (freed 0x55a1c1234)

💡 **The Rule of Three:** If your class needs a custom destructor, copy constructor, OR copy assignment operator — it almost certainly needs all three. A class managing raw memory needs all three to be correct.

Rule of Three (in C++):

If you define any of:

1. Destructor
2. Copy Constructor
3. Copy Assignment Operator

You should define ALL THREE.

(In modern C++11+, this extends to the "Rule of Five"
adding: Move Constructor + Move Assignment Operator)

13.2 Destructors

A **destructor** is the cleanup counterpart to the constructor. It is called automatically when an object:

- Goes out of scope (reaches the end of a block })
- Is explicitly delete d (for heap-allocated objects)

DESTRUCTOR RULES:

- Same name as class, prefixed with tilde: ~ClassName()
- No parameters, no return type
- Only ONE destructor per class (cannot be overloaded)
- Called in REVERSE order of construction

```

#include <stdio.h>
#include <stdlib.h>

class FileHandle {
private:
    char filename[50];
    char* buffer;
    int bufferSize;

public:
    FileHandle(const char* name, int bufSize) {
        strncpy(filename, name, 49);
        bufferSize = bufSize;
        buffer = (char*)malloc(bufSize);    // Allocate memory
        printf("FileHandle opened: %s (buffer=%d bytes)\n",
            filename, bufferSize);
    }

    // ——— DESTRUCTOR —————
    // Automatically called when object goes out of scope.
    // CRUCIAL: free any resources acquired in the constructor!
    ~FileHandle() {
        if (buffer != NULL) {
            free(buffer);    // Free heap memory
            buffer = NULL;    // Prevent dangling pointer
        }
        printf("FileHandle closed: %s (buffer freed)\n", filename);
        // If we opened an actual file, we'd fclose() it here too
    }

    void write(const char* data) {
        printf("[%s] Writing: %s\n", filename, data);
    }
};

void processFile() {
    printf("--- processFile() called ---\n");
    FileHandle fh("report.txt", 1024);    // Constructor called
    fh.write("Line 1: Hello World");
    fh.write("Line 2: Data here");
    printf("--- processFile() returning ---\n");
} // ← DESTRUCTOR automatically called here!

```

```

int main() {
    printf("=== Destructor Demo ===\n\n");

    processFile();    // Constructor + Destructor inside

    printf("\n--- Heap object ---\n");
    FileHandle* fh2 = new FileHandle("log.txt", 512);
    fh2->write("Log entry 1");
    delete fh2;        // ← DESTRUCTOR called explicitly by delete
    // Without delete: MEMORY LEAK! Buffer never freed.

    printf("\nMain done.\n");
    return 0;
}

```

Output:

```

=== Destructor Demo ===

--- processFile() called ---
FileHandle opened: report.txt (buffer=1024 bytes)
[report.txt] Writing: Line 1: Hello World
[report.txt] Writing: Line 2: Data here
--- processFile() returning ---
FileHandle closed: report.txt (buffer freed)

--- Heap object ---
FileHandle opened: log.txt (buffer=512 bytes)
[log.txt] Writing: Log entry 1
FileHandle closed: log.txt (buffer freed)

Main done.

```

Destruction Order

```
int main() {  
    ClassA a;    // Constructed 1st  
    ClassB b;    // Constructed 2nd  
    ClassC c;    // Constructed 3rd  
    // ...  
}  
// Destructors called in REVERSE: c, then b, then a  
// This is like a stack: last constructed = first destroyed.
```

13.3 this Pointer

The **this pointer** is a hidden pointer that every non-static member function receives. It points to the object on which the function was called.

When you call: `alice.deposit(500);`

C++ translates this internally to:
`BankAccount::deposit(&alice, 500);`

Inside `deposit()`:
`this == &alice` ← 'this' points to the calling object
`this->balance` ← same as `alice.balance`

Why this Is Needed

PROBLEM 1: Parameter name shadows member variable

```
class Person {
    char name[50];
public:
    void setName(char name[]) { // ← 'name' shadows 'this->name'!
        name = name;    // WRONG: assigns param to itself, does nothing
    }
};
```

FIX using 'this':

```
void setName(char name[]) {
    strcpy(this->name, name); // this->name = member, name = param
}
```

PROBLEM 2: Returning the current object for method chaining

Without 'this': builder.setX(1); builder.setY(2); builder.setZ(3);

With 'this': builder.setX(1).setY(2).setZ(3); ← Method chaining!

```

#include <stdio.h>
#include <string.h>

class QueryBuilder {
private:
    char table[30];
    char condition[100];
    int limitVal;
    int hasCondition;
    int hasLimit;

public:
    QueryBuilder() : limitVal(0), hasCondition(0), hasLimit(0) {
        table[0] = condition[0] = '\0';
    }

    // — 'this' SOLVES NAMING CONFLICT —————
    QueryBuilder& from(const char* table) {
        // 'table' (parameter) vs this->table (member)
        strncpy(this->table, table, 29);
        return *this;    // Return reference to this object
    }

    // — METHOD CHAINING via 'this' —————
    QueryBuilder& where(const char* condition) {
        strncpy(this->condition, condition, 99);
        hasCondition = 1;
        return *this;    // ← returning *this enables chaining
    }

    QueryBuilder& limit(int limitVal) {
        this->limitVal = limitVal;    // disambiguate
        hasLimit = 1;
        return *this;
    }

    void build() const {
        printf("SELECT * FROM %s", table);
        if (hasCondition) printf(" WHERE %s", condition);
        if (hasLimit)     printf(" LIMIT %d", limitVal);
        printf(";\n");
    }
}

```

```
// — COMPARING OBJECTS USING 'this' —————
bool isSameAs(const QueryBuilder& other) const {
    // 'this' lets us explicitly compare the two objects
    return (this == &other);    // Are they literally the same object?
}

};

int main() {
    printf("=== this Pointer Demo ===\n\n");

    // Method chaining – returns *this from each method
    QueryBuilder qb;
    qb.from("users")
        .where("age > 18")
        .limit(10)
        .build();

    // Second query – different configuration
    QueryBuilder qb2;
    qb2.from("products")
        .where("price < 100.0")
        .build();

    // Self-comparison using this
    printf("\nqb same as qb?  %s\n", qb.isSameAs(qb)  ? "YES" : "NO");
    printf("qb same as qb2? %s\n", qb.isSameAs(qb2) ? "YES" : "NO");

    return 0;
}
```

Output:

```
=== this Pointer Demo ===
```

```
SELECT * FROM users WHERE age > 18 LIMIT 10;
```

```
SELECT * FROM products WHERE price < 100.0;
```

```
qb same as qb?  YES
```

```
qb same as qb2? NO
```

Summary of this Uses

USES OF 'this' POINTER
1. Disambiguate member vs parameter with same name <code>this->name = name;</code>
2. Return the current object for method chaining <code>return *this;</code>
3. Pass the current object to another function <code>someFunction(this);</code>
4. Check if two pointers point to the same object <code>if (this == &other) return *this; // self-assign check</code>
5. Delete the current object (rare, use carefully) <code>delete this;</code>

13.4 Static Variables in C and C++

Static Variables in C

In C, `static` has two completely different meanings depending on where it is used:

1. Static Local Variables — Persistent State

```
#include <stdio.h>

// A static local variable PERSISTS between function calls.
// It is initialized ONCE and retains its value.
int counter() {
    static int count = 0;    // ← 'static': initialized once, lives forever
    count++;
    return count;
}

void callCounterTest() {
    printf("Call 1: %d\n", counter());    // 1
    printf("Call 2: %d\n", counter());    // 2
    printf("Call 3: %d\n", counter());    // 3
    // 'count' is NOT reset each call!
}

// WITHOUT static:
int badCounter() {
    int count = 0;    // Reset to 0 every call!
    count++;
    return count;    // Always returns 1 – useless as a counter
}
```

2. Static Global Variables/Functions — File Scope

```
// In file: math_utils.c
static float pi = 3.14159f;    // Only visible in THIS file
                                // Other files cannot access it

static float circleArea(float r) {    // Also file-scoped
    return pi * r * r;
}

float publicArea(float r) {    // No static: visible from other files
    return circleArea(r);
}
```

3. Static in C++ — Same C Rules + Class Context

STATIC KEYWORD SUMMARY:

Location	Meaning
Inside function	Variable persists across calls (not stack)
Global/file level	Limits visibility to this translation unit
In a class (member)	Belongs to the class, shared by all objects
In a class (function)	Can be called without an object instance

```

#include <stdio.h>

// — STATIC LOCAL: persists between calls —————
int generateID() {
    static int nextID = 1000;    // Initialized ONCE when first called
    return nextID++;
}

// — STATIC vs NON-STATIC LOCAL comparison —————
void compareStatic() {
    static int persistent = 0;    // Survives between calls
    int        temporary  = 0;    // Reset each call

    persistent++;
    temporary++;

    printf("persistent=%d, temporary=%d\n", persistent, temporary);
}

int main() {
    printf("=== Static Variable Demo ===\n\n");

    printf("IDs: %d, %d, %d\n",
           generateID(), generateID(), generateID());
    // Output: IDs: 1000, 1001, 1002

    printf("\n");
    compareStatic(); // persistent=1, temporary=1
    compareStatic(); // persistent=2, temporary=1 ← temporary always 1!
    compareStatic(); // persistent=3, temporary=1

    return 0;
}

```

13.5 Static Members in a Class

Static class members are **shared among all objects** of that class. They belong to the class itself, not to any individual object.

NON-STATIC (each object has its own):

Object A: balance = \$500

Object B: balance = \$1200

Object C: balance = \$750

STATIC (shared by all):

All objects share:

objectCount = 3

Static Data Member Rules

```
class Robot {
private:
    int id;
    char model[30];

    // — STATIC DATA MEMBER: declared here —————
    static int totalRobots;      // Declaration (not definition!)
    static int nextId;

public:
    Robot(const char* m) {
        strncpy(model, m, 29);
        totalRobots++;
        id = nextId++;
        printf("Robot #%d (%s) online. Total: %d\n", id, model, totalRobots);
    }

    ~Robot() {
        totalRobots--;
        printf("Robot #%d (%s) offline. Total: %d\n", id, model, totalRobots);
    }

    // — STATIC MEMBER FUNCTION —————
    // No 'this' pointer. Can only access static members.
    static int getTotalRobots() { return totalRobots; }
    static void reset()        { totalRobots = 0; nextId = 1; }

    // — REGULAR MEMBER FUNCTION —————
    void status() const {
        printf("Robot #%d (%s) | Fleet size: %d\n",
            id, model, totalRobots);
    }
};

// — MUST DEFINE static members OUTSIDE the class —————
// Without this line: linker error ("undefined reference")!
int Robot::totalRobots = 0;
int Robot::nextId     = 1;

int main() {
    printf("=== Static Members Demo ===\n\n");
```

```

// Access static member via class name (no object needed!)
printf("Initial fleet: %d\n\n", Robot::getTotalRobots());

Robot r1("Terminator-T800");
Robot r2("WALL-E");

{
    // Inner scope
    Robot r3("R2-D2");
    r3.status();
    printf("Fleet inside scope: %d\n\n", Robot::getTotalRobots());
} // r3 destructor called here → totalRobots decremented

printf("Fleet after scope: %d\n", Robot::getTotalRobots());

r1.status();
r2.status();

return 0;
}

```

Output:

=== Static Members Demo ===

Initial fleet: 0

Robot #1 (Terminator-T800) online. Total: 1

Robot #2 (WALL-E) online. Total: 2

Robot #3 (R2-D2) online. Total: 3

Robot #3 (R2-D2) | Fleet size: 3

Fleet inside scope: 3

Robot #3 (R2-D2) offline. Total: 2

Fleet after scope: 2

Robot #1 (Terminator-T800) | Fleet size: 2

Robot #2 (WALL-E) | Fleet size: 2

Static Constant Members

```
class Circle {
public:
    // Static const: value known at compile time, shared by all
    static const float PI;           // Declaration
    static const int   MAX_RADIUS;   // Declaration

    float radius;

    Circle(float r) : radius(r) {}
    float area()      const { return PI * radius * radius; }
    float circumference() const { return 2.0f * PI * radius; }
};

// Definition (for non-integer types, must be outside class)
const float Circle::PI      = 3.14159265f;
const int   Circle::MAX_RADIUS = 10000;
```

Task 2

Build a Fleet management system using static members.

Requirements:

1. Create a `Vehicle` class with: make, model, year, mileage, and a unique auto-incremented ID.
2. Use static members: `totalVehicles` , `nextVehicleID` , `totalMileage` (sum of all vehicles).
3. Implement: Constructor (auto-assigns ID), Destructor (updates counters), `drive(float km)` (updates mileage).
4. Add static functions: `getFleetSize()` , `getAverageMileage()` , `printFleetStats()` .
5. Demonstrate with 4 vehicles: 2 on heap (deleted mid-program), 2 on stack.

Bonus: Add a static `Vehicle* mostDriven` pointer that always points to the vehicle with the highest mileage. Update it in `drive()` .

14.0 Friend Functions and Friend Classes

Learning Objectives

- Understand what a friend is and when to use it
- Implement friend functions that access private members of a class
- Implement friend classes where all members of one class can access another
- Recognize the trade-offs and correct use cases for friend declarations

Introduction: Breaking Encapsulation — Carefully

Encapsulation says: *only member functions can access private data*. But sometimes two independent classes need to work together so closely that strict encapsulation becomes an obstacle.

The `friend` keyword **selectively grants** one specific function or class permission to access private members — without making everything public.

WITHOUT FRIEND:

```
class Box {
    int volume;    // private
};

class BoxPrinter {
    void print(Box b) {
        // ERROR: volume is private!
    }
};
```

WITH FRIEND:

```
class Box {
    int volume;    // private
    friend class BoxPrinter; ← grants access
};

class BoxPrinter {
    void print(Box b) {
        // OK: friend access!
        printf("%d\n", b.volume);
    }
};
```

FRIEND DECLARATION RULES:

- ✓ Declared **INSIDE** the class granting access
- ✓ The class (not the friend) decides who is a friend
- ✓ Friendship is NOT reciprocal (A befriends B $\neq$ B befriends A)
- ✓ Friendship is NOT inherited (A's friend $\neq$ A's child's friend)
- ✓ Friend functions are NOT member functions of the class

14.1 Friend Functions

A **friend function** is an ordinary (non-member) function that has been granted access to the private and protected members of a class.

Use Case 1: Operator Overloading

The most common reason for friend functions — enabling operators like `+`, `-`, `<<` to work with your class:

```

#include <stdio.h>

class Complex {
private:
    float real;
    float imag;

public:
    Complex(float r = 0.0f, float i = 0.0f) : real(r), imag(i) {}

    // — FRIEND FUNCTION: accesses private real and imag —————
    // Declared inside Complex, but defined outside (it's not a member)
    friend Complex addComplex(const Complex& a, const Complex& b);
    friend void    printComplex(const Complex& c);
    friend float   magnitude(const Complex& c);

    // Getters (alternative to friend, but friend is more direct here)
    float getReal() const { return real; }
    float getImag() const { return imag; }
};

// — FRIEND FUNCTION DEFINITIONS —————
// These are NOT member functions – no Complex:: prefix
// But they CAN access private members because they are friends

Complex addComplex(const Complex& a, const Complex& b) {
    // Direct access to private members real and imag
    return Complex(a.real + b.real,    // ← private access allowed!
                  a.imag + b.imag);
}

void printComplex(const Complex& c) {
    if (c.imag >= 0)
        printf("%.2f + %.2fi", c.real, c.imag);
    else
        printf("%.2f - %.2fi", c.real, -c.imag);
}

float magnitude(const Complex& c) {
    return (float)(c.real * c.real + c.imag * c.imag); // simplified
}

int main() {

```

```

printf("=== Friend Function Demo ===\n\n");

Complex c1(3.0f, 4.0f);    // 3 + 4i
Complex c2(1.0f, -2.0f);   // 1 - 2i

printf("c1 = "); printComplex(c1); printf("\n");
printf("c2 = "); printComplex(c2); printf("\n");

Complex c3 = addComplex(c1, c2);
printf("c1 + c2 = "); printComplex(c3); printf("\n");

printf("|c1| = %.2f\n", magnitude(c1));

return 0;
}

```

Output:

```

=== Friend Function Demo ===

c1 = 3.00 + 4.00i
c2 = 1.00 - 2.00i
c1 + c2 = 4.00 + 2.00i
|c1| = 25.00

```

Use Case 2: Accessing Two Private Classes

Friend functions shine when bridging two classes that need to cooperate:

```

#include <stdio.h>

class Rectangle;    // Forward declaration (tell compiler Rectangle exists)

class Square {
private:
    float side;
public:
    Square(float s) : side(s) {}

    // Friend function needs access to BOTH Square and Rectangle
    friend bool canFitInside(const Square& sq, const Rectangle& rect);
};

class Rectangle {
private:
    float width;
    float height;
public:
    Rectangle(float w, float h) : width(w), height(h) {}

    // Same friend function granted access here too
    friend bool canFitInside(const Square& sq, const Rectangle& rect);
};

// This ONE function can access private members of BOTH classes
bool canFitInside(const Square& sq, const Rectangle& rect) {
    return (sq.side <= rect.width && sq.side <= rect.height);
}

int main() {
    Square sq(5.0f);
    Rectangle r1(10.0f, 8.0f);
    Rectangle r2(4.0f, 3.0f);

    printf("Square(5) fits in Rectangle(10x8)? %s\n",
           canFitInside(sq, r1) ? "YES" : "NO");
    printf("Square(5) fits in Rectangle(4x3)? %s\n",
           canFitInside(sq, r2) ? "YES" : "NO");

    return 0;
}

```

Use Case 3: Friend with a Specific Class Method

You can befriend a SPECIFIC method of another class (not the whole class):

```
class Wallet;

class Inspector {
public:
    // Only THIS method gets friend access to Wallet
    void auditWallet(const Wallet& w);
    void doSomethingElse() { /* no Wallet access */ }
};

class Wallet {
private:
    float balance;
    int transactionCount;

public:
    Wallet(float b) : balance(b), transactionCount(0) {}

    // Grant access to ONLY Inspector::auditWallet
    friend void Inspector::auditWallet(const Wallet& w);
};

void Inspector::auditWallet(const Wallet& w) {
    // Has access to private balance and transactionCount
    printf("Audit: balance=%.2f, transactions=%d\n",
        w.balance, w.transactionCount);
}
```

14.2 Friend Classes

When an **entire class** is declared as a friend, **all member functions** of the friend class can access private members of the granting class.

When to Use Friend Classes

Use friend classes when:

- ✓ Two classes are fundamentally coupled (e.g., `List` and `ListIterator`)
- ✓ A "helper" class needs deep access to implement a complex operation
- ✓ Testing: a test class needs access to internals for verification
- ✓ Serialization: a serializer class reads all private fields

Avoid friend classes when:

- X A simple getter/setter would suffice
- X The classes have no natural intimate relationship
- X You're trying to "work around" bad design

Example 1: `LinkedList` and `Iterator`

The classic use case — an iterator needs intimate access to list internals:

```

#include <stdio.h>
#include <stdlib.h>

// — FORWARD DECLARATIONS —————
class LinkedList;

// — NODE (private to LinkedList logic) —————
struct Node {
    int    data;
    Node*  next;
    Node(int d) : data(d), next(nullptr) {}
};

// — ITERATOR CLASS —————
// Provides a way to traverse LinkedList without exposing its internals
class ListIterator {
private:
    Node* current;    // Points to current node during iteration

public:
    ListIterator(Node* start) : current(start) {}

    bool hasNext() const { return current != nullptr; }

    int next() {
        int val = current->data;
        current = current->next;
        return val;
    }
};

// — LINKED LIST CLASS —————
class LinkedList {
private:
    Node* head;
    Node* tail;
    int    size;

    // — FRIEND DECLARATION: ListIterator gets full access —————
    friend class ListIterator;

public:
    LinkedList() : head(nullptr), tail(nullptr), size(0) {}

```

```

~LinkedList() {
    Node* curr = head;
    while (curr) {
        Node* next = curr->next;
        delete curr;
        curr = next;
    }
}

void append(int val) {
    Node* n = new Node(val);
    if (!tail) { head = tail = n; }
    else { tail->next = n; tail = n; }
    size++;
}

// Factory: returns an iterator starting at head
ListIterator getIterator() const {
    return ListIterator(head);
}

int getSize() const { return size; }
};

int main() {
    printf("=== Friend Class Demo - LinkedList + Iterator ===\n\n");

    LinkedList list;
    list.append(10);
    list.append(20);
    list.append(30);
    list.append(40);
    list.append(50);

    printf("List (%d elements): ", list.getSize());

    // Iterate using the friend class
    ListIterator it = list.getIterator();
    while (it.hasNext()) {
        printf("%d ", it.next());
    }
    printf("\n");
}

```

```
    return 0;  
}
```

Example 2: Matrix and MatrixHelper

A computation helper that needs deep access to Matrix internals:

```

#include <stdio.h>
#include <stdlib.h>

class MatrixHelper;    // Forward declaration

class Matrix {
private:
    float** data;
    int     rows;
    int     cols;

    // — FRIEND CLASS: MatrixHelper can access everything —————
    friend class MatrixHelper;

public:
    Matrix(int r, int c) : rows(r), cols(c) {
        data = (float**)malloc(r * sizeof(float*));
        for (int i = 0; i < r; i++)
            data[i] = (float*)calloc(c, sizeof(float));
    }

    ~Matrix() {
        for (int i = 0; i < rows; i++) free(data[i]);
        free(data);
    }

    void set(int r, int c, float val) { data[r][c] = val; }
    float get(int r, int c) const { return data[r][c]; }

    void print() const {
        printf("Matrix %dx%d:\n", rows, cols);
        for (int i = 0; i < rows; i++) {
            printf("  [");
            for (int j = 0; j < cols; j++)
                printf(" %6.2f", data[i][j]);
            printf(" ]\n");
        }
    }
};

class MatrixHelper {
public:
    // Adds two matrices: result = a + b

```

```

static Matrix add(const Matrix& a, const Matrix& b) {
    // Friend access: can read a.rows, a.cols, a.data directly
    if (a.rows != b.rows || a.cols != b.cols) {
        printf("Error: dimension mismatch!\n");
        return Matrix(0, 0);
    }

    Matrix result(a.rows, a.cols);
    for (int i = 0; i < a.rows; i++)
        for (int j = 0; j < a.cols; j++)
            result.data[i][j] = a.data[i][j] + b.data[i][j];

    return result;
}

// Transposes a matrix (rows become columns)
static Matrix transpose(const Matrix& m) {
    Matrix result(m.cols, m.rows);
    for (int i = 0; i < m.rows; i++)
        for (int j = 0; j < m.cols; j++)
            result.data[j][i] = m.data[i][j];
    return result;
}

// Computes trace (sum of diagonal)
static float trace(const Matrix& m) {
    int limit = (m.rows < m.cols) ? m.rows : m.cols;
    float sum = 0.0f;
    for (int i = 0; i < limit; i++)
        sum += m.data[i][i];
    return sum;
}

};

int main() {
    printf("=== Friend Class Demo – Matrix + MatrixHelper ===\n\n");

    Matrix a(2, 2), b(2, 2);
    a.set(0,0, 1); a.set(0,1, 2);
    a.set(1,0, 3); a.set(1,1, 4);

    b.set(0,0, 5); b.set(0,1, 6);
    b.set(1,0, 7); b.set(1,1, 8);

```

```

    printf("Matrix A:"); a.print();
    printf("Matrix B:"); b.print();

    Matrix sum = MatrixHelper::add(a, b);
    printf("A + B:");    sum.print();

    Matrix t = MatrixHelper::transpose(a);
    printf("A transposed:"); t.print();

    printf("Trace of A: %.2f\n", MatrixHelper::trace(a));

    return 0;
}

```

Output:

=== Friend Class Demo – Matrix + MatrixHelper ===

Matrix A:

```

[  1.00  2.00 ]
[  3.00  4.00 ]

```

Matrix B:

```

[  5.00  6.00 ]
[  7.00  8.00 ]

```

A + B:

```

[  6.00  8.00 ]
[ 10.00 12.00 ]

```

A transposed:

```

[  1.00  3.00 ]
[  2.00  4.00 ]

```

Trace of A: 5.00

Friend: Complete Reference

FRIEND – COMPLETE REFERENCE	
Syntax	What it grants
<code>friend void func(A&);</code>	<code>func()</code> can access A's private members
<code>friend class B;</code>	ALL methods of B can access A's privates
<code>friend void B::method();</code>	ONLY <code>B::method()</code> can access A's privates
PROPERTIES:	
X NOT reciprocal: A befriends B $\neq$ B befriends A	
X NOT inherited: A's friend $\neq$ child of A's friend	
X NOT transitive: $A \rightarrow B$ friends, $B \rightarrow C$ friends $\neq$ $A \rightarrow C$ friends	
✓ Friend has access to ALL private/protected members	
✓ The CLASS decides who its friends are (inside the class)	
WHEN TO USE:	
✓ Operator overloading (<code><<</code> , <code>>></code> , <code>+</code> , <code>-</code> , <code>==</code>)	
✓ Tightly coupled pairs (List/Iterator, Matrix/MatrixOps)	
✓ Unit testing internals	
✓ Serialization/deserialization helpers	
X DON'T use to bypass poor design – redesign instead	

Common OOP Mistakes — Full Chapter Summary

CHAPTER 12:

- ✗ Making everything public "for convenience" → defeats encapsulation
- ✗ Using inheritance when composition is more appropriate
(prefer composition over inheritance as a rule of thumb)
- ✗ Forgetting 'virtual' on the destructor of a base class
→ Derived class destructor never called → memory leak!
Fix: always declare virtual ~BaseClass() in polymorphic bases.
- ✗ Confusing IS-A (inherit) with HAS-A (compose)

CHAPTER 13:

- ✗ Defining a parameterized constructor but not a default one
→ MyClass arr[10]; fails (no default constructor!)
- ✗ Shallow copy of pointer members (use deep copy for raw pointers)
- ✗ Forgetting to free() in destructor → memory leak
- ✗ Double free in shallow copy → crash
- ✗ Self-assignment in operator=:
Fix: always check if (this == &other) return *this; first.
- ✗ Using 'this' in a static member function → no 'this' in static!

CHAPTER 14:

- ✗ Overusing friend – if you need lots of friends, redesign
- ✗ Assuming friendship is two-way (it isn't)
- ✗ Declaring friend in the wrong class (friendship is granted BY the class being accessed, not BY the friend)

Best Practices — Chapters 12–14

DESIGN:

- ✓ Prefer 'private' for data, 'public' for interface (encapsulate first)
- ✓ Use 'const' on every member function that doesn't modify the object
- ✓ Prefer composition over inheritance
- ✓ Design classes around what they DO, not what they ARE

CONSTRUCTORS:

- ✓ Always use member initializer lists (: member(value))
- ✓ Mark single-argument constructors 'explicit' to prevent accidental conversions
- ✓ Follow the Rule of Three: destructor + copy ctor + copy assignment = define all 3
- ✓ Always initialize ALL member variables in the constructor

DESTRUCTORS:

- ✓ Always declare base class destructor as 'virtual'
- ✓ Always free every resource acquired (malloc → free, new → delete)
- ✓ Set freed pointers to NULL after freeing

FRIEND:

- ✓ Use friend sparingly – it weakens encapsulation
- ✓ Prefer public getters/setters when possible
- ✓ When you do use friend, document WHY clearly

Chapter 12–14 — Final Exercises

Exercise 1 (Beginner): Create a `Rectangle` class with private `width` and `height`. Add all four constructors (default, parameterized, copy). Print a message in each constructor and the destructor to observe the lifecycle.

Exercise 2 (Intermediate): Implement a `String` class that wraps a dynamically allocated `char*`. Implement deep copy constructor, copy assignment, and destructor. Test by creating, copying, modifying one copy, and verifying the original is unchanged.

Exercise 3 (Advanced): Design a `vector` class (like `std::vector`) using dynamic array:

- Private: `int* data`, `int size`, `int capacity`
- Constructors: default (`capacity=2`), parameterized (initial size)

- `push_back(int val)` — doubles capacity when full
- Deep copy constructor and assignment
- Destructor
- `operator[]` for access
- Friend function `printVector(const Vector& v)`



Chapters 12–14 Quick Reference

OOP COMPLETE QUICK REFERENCE			
CLASS MEMBERS	ACCESS	CAN USE 'this'?	SHARED?
Regular method	all	YES	per-object
const method	all	YES (read-only)	per-object
static method	static only	NO	class-wide
virtual method	all	YES	overridable
CONSTRUCTORS	CALLED WHEN		
Default ()	MyClass obj;		
Parameterized (a,b)	MyClass obj(a, b);		
Copy (const C& o)	MyClass obj2 = obj; / func(obj); / return		
COPY TYPE	POINTERS COPIED	SAFE?	USE WHEN
Shallow	Address only	NO*	No pointer members
Deep	New alloc + copy	YES	Has pointer members
*Causes double-free and aliasing bugs			
FRIEND	DECLARED WHERE	GRANTS	
friend func	In granting class	One function's access	
friend class	In granting class	All methods' access	

Additional Resources

Books

- **"The C++ Programming Language" by Bjarne Stroustrup** — Written by the creator of C++. The definitive reference for OOP in C++.
- **"Effective C++" by Scott Meyers** — 55 specific rules for writing excellent C++. Read after the basics.
- **"Object-Oriented Analysis and Design with Applications" by Grady Booch** — Deep dive into OOP thinking and design.
- **"Design Patterns" by Gang of Four (GoF)** — 23 classic OOP design patterns. The Bible of OOP architecture.

Online Resources

- learncpp.com — The best free online C++ tutorial. Covers every concept in this chapter with examples.
- cppreference.com — Authoritative C++ language reference.
- isocpp.org — The C++ Standards Committee's website. C++ Core Guidelines here are essential reading.

Tools

- **GCC / Clang with `-Wall -Wextra`** — Enable all warnings. Your compiler will catch many OOP mistakes.
- **Valgrind** — Detect memory leaks, double frees, and uninitialized reads.
- **AddressSanitizer** (`-fsanitize=address`) — Built-in memory error detector.

 **Final Pro Tip:** The single most important OOP principle for beginners to internalize is **encapsulate first, relax later**. Start with everything `private`. Then ask: "what does the outside world absolutely need to see?" Make only that public. It's far easier to relax access restrictions later than to tighten them after other code has started depending on your internals.

